

第 11 章：数据库设计

2026 版

邹兆年

哈尔滨工业大学
计算学部
海量数据计算研究中心
电子邮件: znzou@hit.edu.cn

2026 春

教学内容¹

- ① 11.1 数据库设计的过程
- ② 11.2 概念数据库设计
 - 11.2.1 与实体相关的概念
 - 11.2.2 与联系相关的概念
- ③ 11.3 逻辑数据库设计
 - 11.3.1 ER 模型转换为关系数据库模式
 - 11.3.2 关系模式规范化
- ④ 11.4 物理数据库设计
 - 11.4.1 工作负载分析
 - 11.4.2 索引设计
 - 11.4.3 查询改写

¹课件更新于 2026 年 6 月 22 日

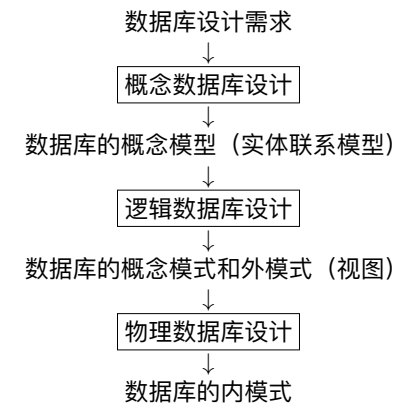
11.1 数据库设计的过程

- 数据库设计：设计数据库模式（本课程的内容）
- 数据库应用设计与开发：设计与开发应用程序（软件工程课程的内容）

数据库设计的过程

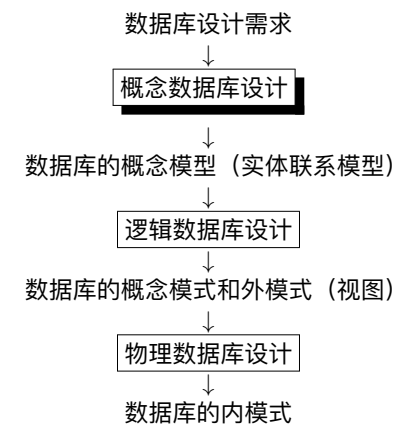
- 1 概念数据库设计 (Conceptual Database Design): 设计数据库的**概念模型**
- 2 逻辑数据库设计 (Logical Database Design): 根据所使用 DBMS 的实现数据模型, 设计数据库的**概念模式**和**外模式**
- 3 物理数据库设计 (Physical Database Design): 根据性能需求和应用访问数据库的特点, 设计数据库的**内模式**

数据库设计的过程



11.2 概念数据库设计

数据库设计的过程



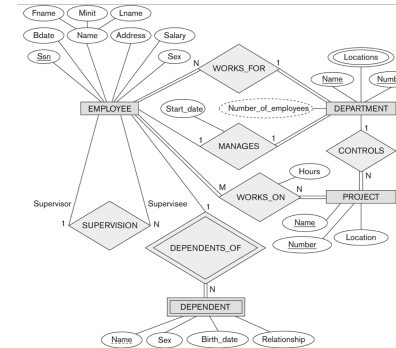
实体-联系模型 (ER 模型, Entity-Relationship Model, E-R Model)

建模实体 (E) 和联系 (R)

- 概念数据库设计阶段使用的一种重要的概念模型
- ER 模型将现实世界抽象表示为实体 (Entity) 及实体间的联系 (Relationship)

实体-联系图 (ER 图, Entity-Relationship Diagram, E-R Diagram)

ER 模型的图形表示



ER 模型提供的概念

与实体相关的概念

- 实体
- 属性
- 键
- 实体型、实体集
- 弱实体型

与联系相关的概念

- 联系
- 联系型、联系集
- 联系型的约束
- 联系型的属性
- 多元联系

数据库设计需求示例

China Manned Space (CMS)



数据库设计需求示例

例 (中国载人航天工程知识数据库设计需求)

- **记录载人航天飞行任务信息**：飞行任务使用的飞船名称、运载火箭、发射时间、返回时间、发射场、乘组、前序任务。一次发射任务只能使用一个运载火箭，只在一个发射场发射，可以有 0-2 个乘组。0 个乘组代表无人；一般情况下，是 1 个乘组；特殊情况下，有 2 个乘组（如神舟二十一号），其中一个出征乘组，另一个是返回乘组。
- **记录运载火箭信息**：运载火箭型号、运力（吨）、服役情况、参与的载人航天飞行任务。一种型号的运载火箭可以在多个发射任务中使用。
- **记录发射场信息**：发射场名称、启用时间、坐标、承担的载人航天飞行任务。坐标包括经度和纬度。
- **记录乘组信息**：乘组由 1-3 名航天员组成，每名航天员在乘组中都有自己的编号（01 号：指令长；02 号：航天飞行工程师；03 号：载荷专家）。记录乘组参与的发射任务、乘组的任务（唯一乘组、出征乘组、返回乘组）。
- **记录航天员信息**：姓名、性别、出生日期、职业、参与的载人航天飞行任务。一名航天员可以有多个职业，可以参加多次飞行任务，在每次任务的乘组中有自己的编号。

11.2.1 与实体相关的概念

实体 (Entity)

定义 (实体)

现实世界中具体的对象或事物

例 (实体)

- 神舟一号载人飞船
- 长征 2F 运载火箭
- 酒泉卫星发射中心
- 杨利伟

属性 (Attribute)

定义 (属性)

用于刻画实体的特性

- 每个属性都有值域
- 一个实体全部属性的值共同刻画了该实体

例 (属性)

- 神舟一号载人飞船的发射时间
- 长征二号 F 运载火箭的服役情况
- 酒泉卫星发射中心的启用时间
- 杨利伟的姓名

属性的分类

- 简单属性
- 复合属性
- 多值属性
- 派生属性

简单属性 (Simple Attribute)

简单属性不可分

定义 (简单属性)

具有原子 (Atomic) 属性值的属性，其属性值不可再分

例 (简单属性)

- 神舟一号载人飞船的名称
- 长征二号 F 运载火箭的服役情况
- 酒泉卫星发射中心的启用时间
- 杨利伟的性别

复合属性 (Composite Attribute)

复合属性可分

定义 (复合属性)

由多个成分构成的属性，其属性值可再分

- 复合属性的成分还可以是复合的

例 (复合属性)

- 地址 (含省, 市, 区, 道路, 门牌号, 邮编)
- 长征 2F 运载火箭的典型运力 (含 LEO、GTO、SSO 运力)
- 酒泉卫星发射中心的坐标 (含经度和纬度)
- 杨利伟的姓名 (含姓和名)

多值属性 (Multi-valued Attribute)

多值属性可以有多个值

定义 (多值属性)

一个实体可以有多个值的属性

例 (多值属性)

- 一个人的电话号码
- 一篇论文的作者
- 复合属性和多值属性可以相互嵌套，例如一个人的教育经历

派生属性 (Derived Attribute)

派生属性是从其他属性派生得到的

定义 (派生属性)

从其他属性派生得到的属性

例 (派生属性)

- 一个人的年龄可以通过其出生日期计算得到
- 一个人的身体质量指数 (Body Mass Index, BMI) 可以通过其身高和体重计算得到

实体型 (Entity Type)

实体的类型

定义 (实体型)

具有相同属性的实体共同具有的类型

定义 (键属性)

同一实体型的任意实体都具有不同值的属性

例 (实体型和键属性)

- 航天员是实体型
- 姓名是航天员实体型的键属性

实体集 (Entity Set)

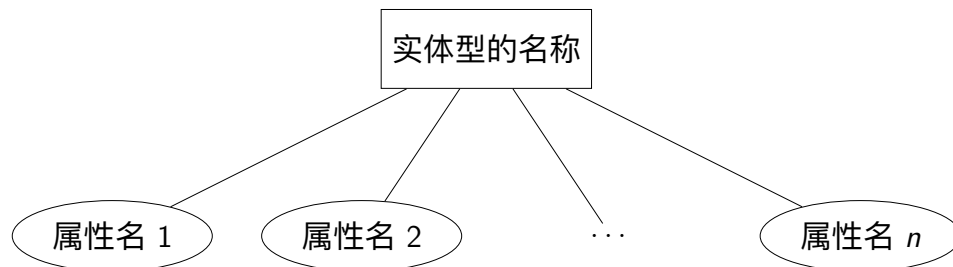
同类实体的集合

定义 (实体集)

当前存储在数据库中的某实体型的实例的集合

实体 $\longleftrightarrow$ 对象
实体型 $\longleftrightarrow$ 对象的类型
实体集 $\longleftrightarrow$ 对象的集合

实体型和属性的 ER 图表示



不同类型属性的 ER 图表示

- 多值属性

多值属性名

- 派生属性

派生属性名

- 键属性

键属性名

- 复合属性

复合属性名

成分 1

成分 2

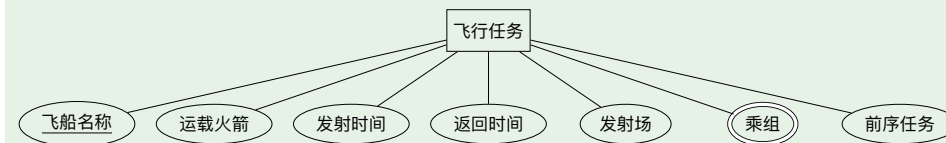
...

成分 n

CMS 知识数据库的初步概念设计

例 (实体型的设计)

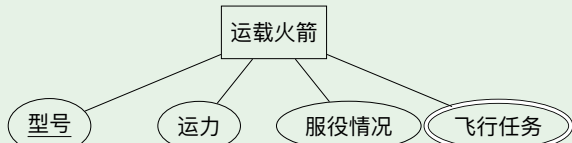
记录载人航天飞行任务信息：飞行任务使用的飞船名称、运载火箭、发射时间、返回时间、发射场、乘组、前序任务。一次发射任务只能使用一个运载火箭，只在一个发射场发射，可以有 0-2 个乘组。0 个乘组代表无人；一般情况下，是 1 个乘组；特殊情况下，有 2 个乘组（如神舟二十一号），其中一个出征乘组，另一个是返回乘组。



CMS 知识数据库的初步概念设计

例 (实体型的设计)

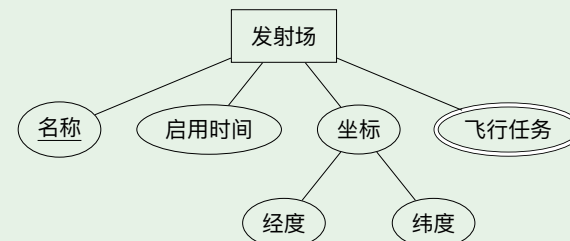
记录运载火箭信息：运载火箭型号、运力（吨）、服役情况、参与的载人航天飞行任务。一种型号的运载火箭可以在多个发射任务中使用。



CMS 知识数据库的初步概念设计

例 (实体型的设计)

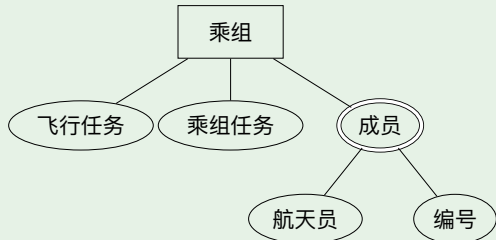
记录发射场信息：发射场名称、启用时间、坐标、承担的载人航天飞行任务。坐标包括经度和纬度。



CMS 知识数据库的初步概念设计

例 (实体型的设计)

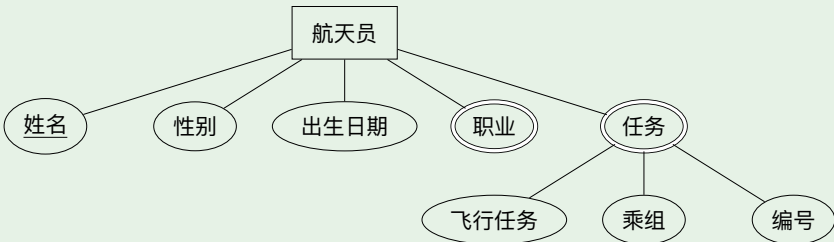
记录乘组信息：乘组由 1-3 名航天员组成，每名航天员在乘组中都有自己的编号（01 号：指令长；02 号：航天飞行工程师；03 号：载荷专家）。记录乘组参与的发射任务、乘组的任务（唯一乘组、出征乘组、返回乘组）。



CMS 知识数据库的初步概念设计

例 (实体型的设计)

记录航天员信息：姓名、性别、出生日期、职业、参与的载人航天飞行任务。一名航天员可以有多个职业，可以参加多次飞行任务，在每次任务的乘组中有自己的编号。



CMS 知识数据库初步概念设计的不足

- 初步设计方案不完善：只刻画了实体型
- 需求中很多内容需要表示为实体间的联系

11.2.2 与联系相关的概念

联系 (Relationship)

定义 (联系)

多个实体之间有意义的关联关系

例 (联系)

- 神舟五号载人飞船是长征二号 F 运载火箭 CZ-2F Y5 的载荷
- 神舟五号载人飞船在酒泉卫星发射中心发射
- 杨利伟参与了神舟五号载人飞船飞行任务

联系型 (Relationship Type)

联系的类型

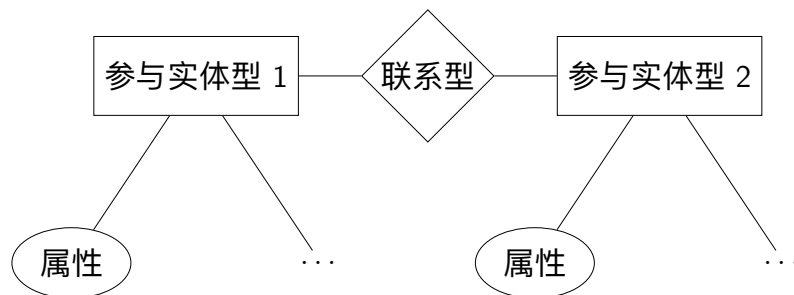
定义 (联系型)

同一类联系共同具有的类型

例 (联系型)

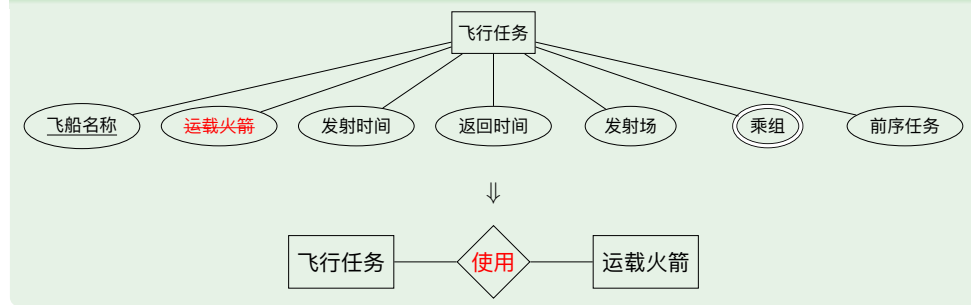
- 载人飞船与运载火箭两个实体型参与的联系型“载荷”
- 载人飞船与发射场两个实体型参与的联系型“发射于”
- 航天员与载人飞船两个实体型参与的联系型“乘组”

联系型的 ER 图表示



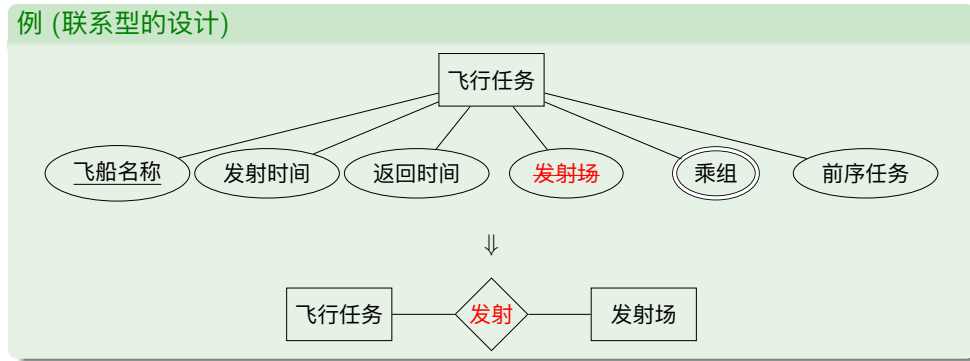
CMS 知识数据库概念设计的细化

例 (联系型的设计)



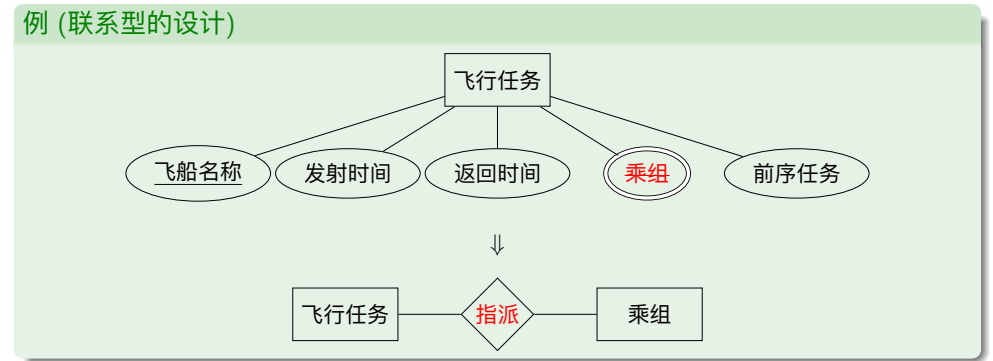
CMS 知识数据库概念设计的细化

例 (联系型的设计)



CMS 知识数据库概念设计的细化

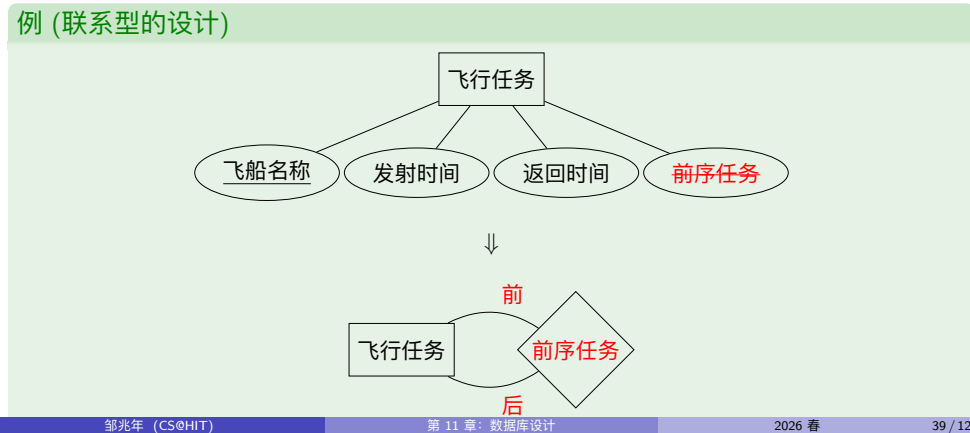
例 (联系型的设计)



CMS 知识数据库概念设计的细化

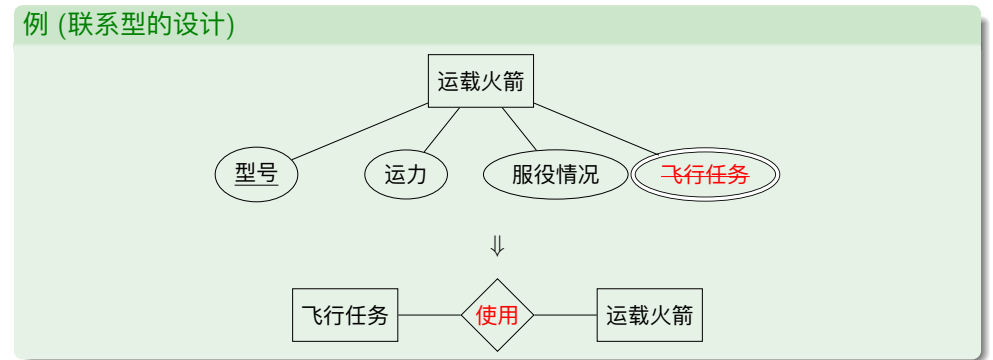
- 一个实体型可以和自身参与到同一个联系型中
- 该实体型在该联系型中担任不同的角色 (Role)，需要在 ER 图中的标出

例 (联系型的设计)



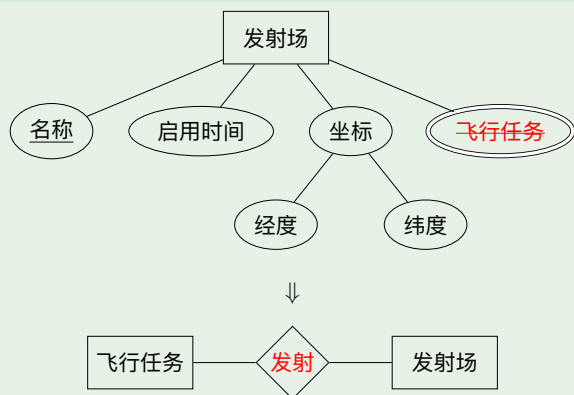
CMS 知识数据库概念设计的细化

例 (联系型的设计)



CMS 知识数据库概念设计的细化

例 (联系型的设计)



弱实体相关概念

没有键属性的实体型

定义 (弱实体型 (Weak Entity Type))

没有键属性的实体型

定义 (标识实体型 (Identifying Entity Type) / 属主实体型 (Owner Entity Type))

由于弱实体型没有键属性, 需要依赖于其他实体型进行区分

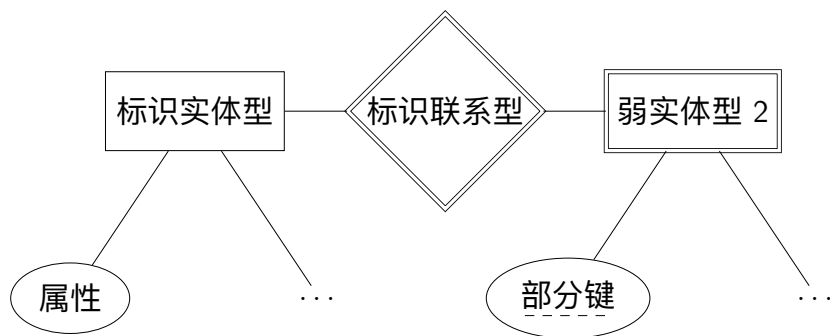
定义 (标识联系型 (Identifying Relationship Type) / 属主联系型 (Owner Relationship Type))

弱实体型与其标识实体型通过标识联系型关联

定义 (部分键 (Partial Key))

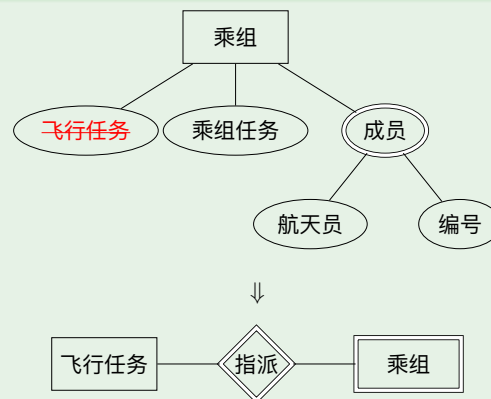
用于区分和同一标识实体相关联的弱实体的属性集合

弱实体型和标识联系型的 ER 图表示



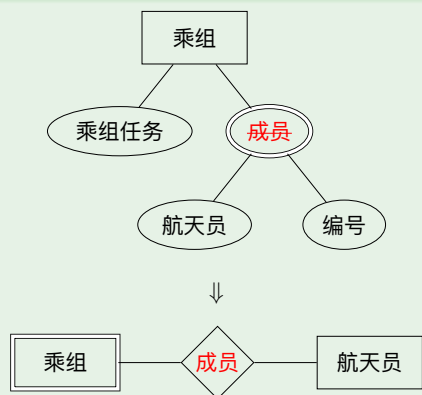
CMS 知识数据库概念设计的细化

例 (弱实体型的设计)



CMS 知识数据库概念设计的细化

例 (弱实体的设计)



联系型的约束

- 基数比 (Cardinality Ratio)
- 参与度

基数比 (Cardinality Ratio)

定义 (基数比)

刻画实体型参与到联系型中的**最大基数**，即一个实体最多参与到多少个联系中

- 多对多 (M:N)
- 多对一 (N:1)
- 一对一 (1:1)

多对多 (M:N) 联系型

定义 (多对多 (M:N) 联系型)

对实体型 1 的任意实体 e ，最多有 $N > 1$ 个实体型 2 的实体与 e 构成联系；对实体型 2 的任意实体 e' ，最多有 $M > 1$ 个实体型 1 的实体与 e' 构成联系



例 (M:N 联系型)



多对一 (N:1) 联系型

定义 (多对一 (N:1) 联系型)

对实体型 1 的任意实体 e ，最多只有 1 个实体型 2 的实体与 e 构成联系；对实体型 2 的任意实体 e' ，最多有 $N > 1$ 个实体型 1 的实体与 e' 构成联系



例 (N:1 联系型)



多对一 (N:1) 联系型

例 (N:1 联系型)



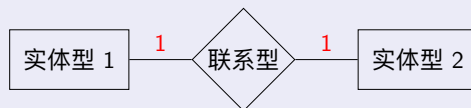
例 (1:N 联系型)



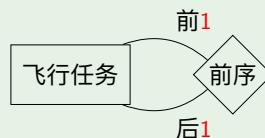
一对一 (1:1) 联系型

定义 (1:1 联系型)

对实体型 1 的任意实体 e ，最多只有 1 个实体型 2 的实体与 e 构成联系；对实体型 2 的任意实体 e' ，最多只有 1 个实体型 1 的实体与 e' 构成联系



例 (1:1 联系型)



存在依赖约束 (Existence Dependency Constraint) / 参与度约束 (Participation Constraint)

定义 (存在依赖约束/参与度约束)

刻画实体型参与到联系型中的**最小基数**，即一个实体最少参与到多少个联系中

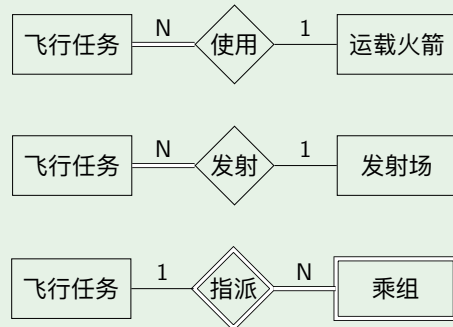
- 0 个 (部分参与): 在 ER 图中表示为单线
- ≥ 1 个 (全部参与): 在 ER 图中表示为双线

例 (参与度)



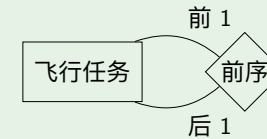
存在依赖约束 (Existence Dependency Constraint) / 参与度约束 (Participation Constraint)

例 (参与度)



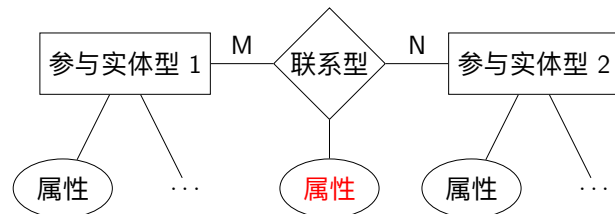
存在依赖约束 (Existence Dependency Constraint) / 参与度约束 (Participation Constraint)

例 (参与度)



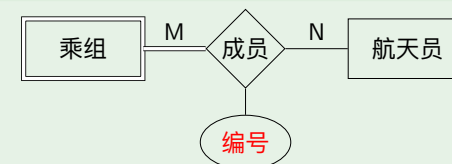
联系型的属性

联系型也可以具有属性



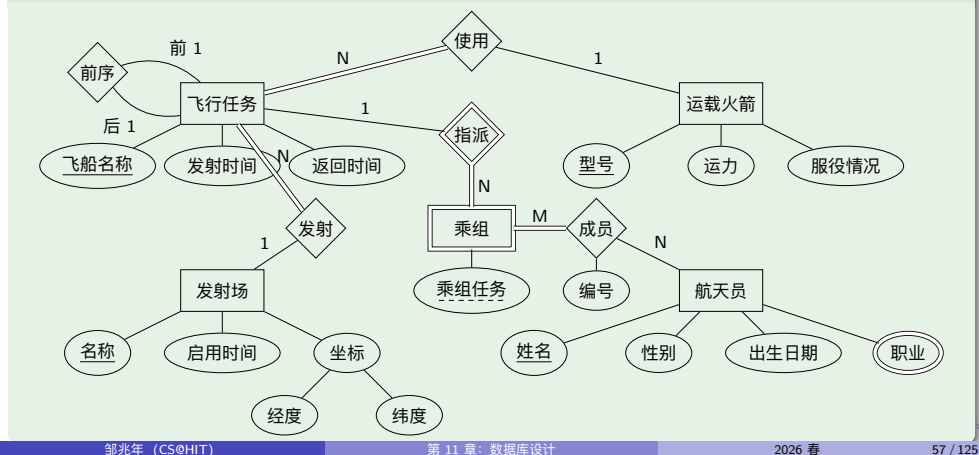
联系型的属性

例 (联系型的属性)



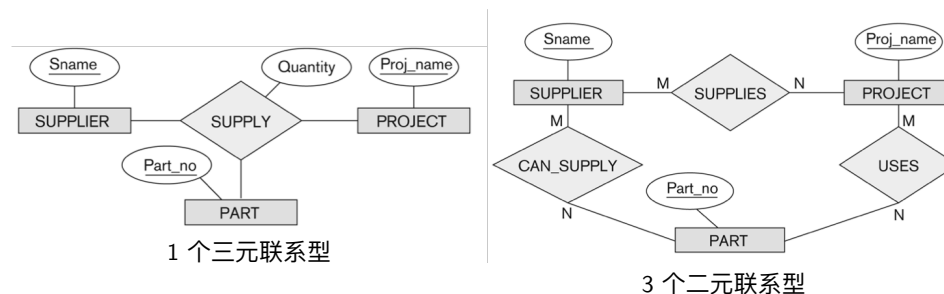
CMS 知识数据库的概念模型

例 (CMS 知识数据库的 ER 图)



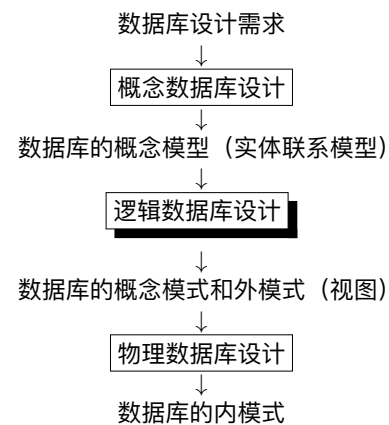
多元联系 (Higher-Degree Relationship)

- 3 个以上实体参与的联系
- 一个 n 元联系和 n 个二元联系所表示的意义通常是不同的



11.3 逻辑数据库设计

数据库设计的过程



11.3.1 ER 模型转换为关系数据库模式

ER 模型转换为关系数据库模式

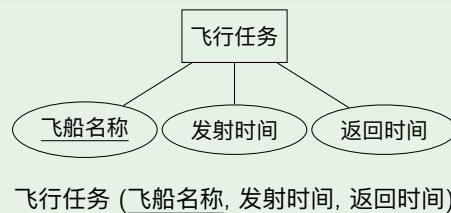
实体型 → 关系模式
联系型 → 关系模式

实体型的转换

转换规则

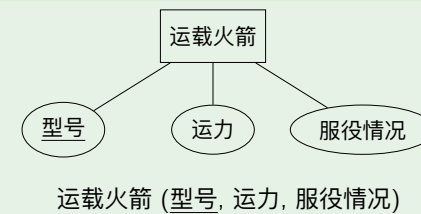
- ① 实体型的名称 → 关系名
- ② 实体型的属性集（多值属性除外）→ 关系的属性集
- ③ 实体型的主键 → 关系的主键
- ④ 实体 → 元组

例 (实体型的转换)



实体型的转换

例 (实体型的转换)

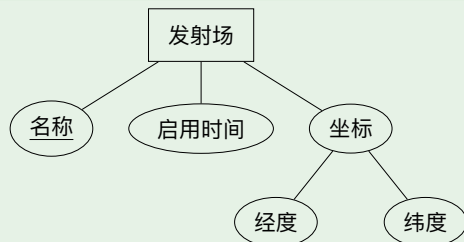


复合属性的转换

转换规则

- ① 复合属性的最低级组成成分 $\rightarrow$ 关系的属性

例 (复合属性的转换)



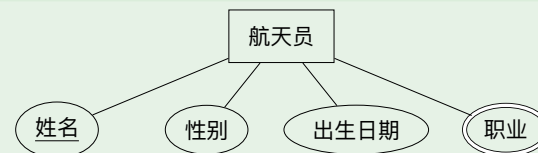
发射场 (名称, 启用时间, 经度, 纬度)

多值属性的转换

转换规则

- ① 实体型关系的属性集中不包含多值属性
- ② 多值属性 $\rightarrow$ 关系
- ③ 将实体型的主键并入多值属性关系中
- ④ 建立多值属性关系到实体型关系的外键约束

例 (多值属性的转换)



航天员 (姓名, 性别, 出生日期)

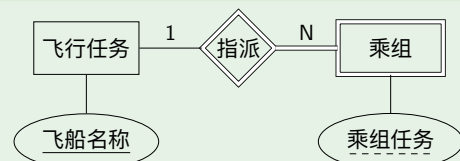
航天员职业 (姓名, 职业)

弱实体型的转换

转换规则

- ① 弱实体型的名称 $\rightarrow$ 关系名
- ② 弱实体型的属性集 $\cup$ 属主实体型的主键 $\rightarrow$ 关系的属性集
- ③ 属主实体型的主键 $\cup$ 弱实体型的部分键 $\rightarrow$ 关系的主键
- ④ 建立弱实体型关系到属主实体型关系的外键约束

例 (弱实体型的转换)



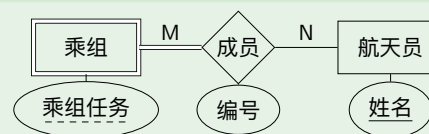
乘组 (飞船名称, 乘组任务)

M:N 联系型的转换

转换规则

- ① 联系型的名称 $\rightarrow$ 关系名
- ② 实体型 1 的主键 $\cup$ 实体型 2 的主键 $\cup$ 联系型的属性集 $\rightarrow$ 关系的属性集
- ③ 实体型 1 的主键 $\cup$ 实体型 2 的主键 $\rightarrow$ 关系的主键
- ④ 建立联系型关系到实体型 1 关系的外键约束
- ⑤ 建立联系型关系到实体型 2 关系的外键约束

例 (M:N 联系型的转换)



成员 (飞船名称, 乘组任务, 姓名, 编号)

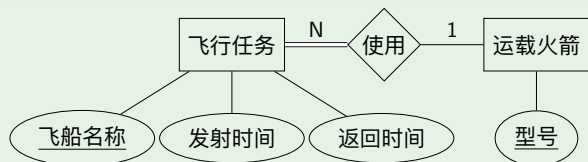
N:1 联系型的转换

转换规则

设联系型中基数为 N 一方的实体型（如飞行任务）为实体型 1，基数为 1 一方的实体型（如运载火箭）为实体型 2

- ① 将实体型 2 的主键和联系型的属性并入实体型 1 的属性集中
- ② 建立实体型 1 关系到实体型 2 关系的外键约束

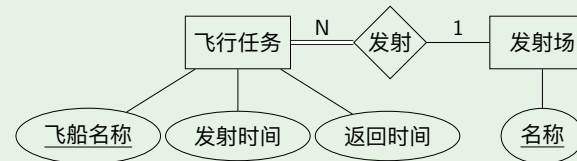
例 (N:1 联系型的转换)



飞行任务 (飞船名称, 发射时间, 返回时间, 运载火箭型号), 其中“运载火箭型号”参照“运载火箭 (型号)”

N:1 联系型的转换

例 (N:1 联系型的转换)



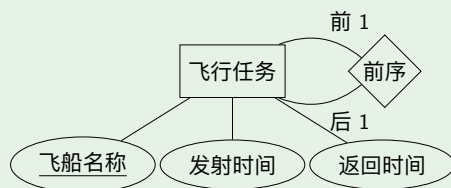
飞行任务 (飞船名称, 发射时间, 返回时间, 发射场名称), 其中“发射场名称”参照“发射场 (名称)”

1:1 联系型的转换

转换规则

与 N:1 联系的转换方法相同

例 (1:1 联系型的转换)

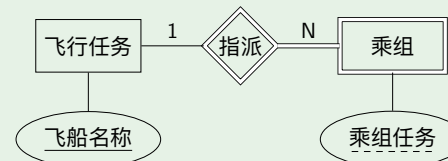


飞行任务 (飞船名称, 发射时间, 返回时间, 前序飞船名称), 其中“前序飞船名称”参照“飞行任务 (飞船名称)”

标识联系型的转换

- 由于弱实体型转换成的关系里已经包含了属主实体型的主键，因此标识联系型就不必转换了

例 (标识联系型的转换)



乘组 (飞船名称, 乘组任务)

CMS 知识数据库模式

实体型和 N:1 (含 1:1) 联系型转换成的关系模式

- 飞行任务 (飞船名称, 发射时间, 返回时间, 运载火箭型号, 发射场名称, 前序飞船名称)
- 运载火箭 (型号, 运力, 服役情况)
- 发射场 (名称, 启用时间, 经度, 纬度)
- 航天员 (姓名, 性别, 出生日期)

弱实体型和标识联系型转换成的关系模式

- 乘组 (飞船名称, 乘组任务)

多值属性转换成的关系模式

- 航天员职业 (姓名, 职业)

M:N 联系型转换成的关系模式

- 成员 (飞船名称, 乘组任务, 姓名, 编号)

11.3.2 关系模式规范化

设计不规范的关系模式可能导致的异常问题

- 数据更新异常：数据插入异常、数据删除异常、数据修改繁琐
- 数据冗余问题：数据重复存储

例 (设计不规范的关系模式)

关系模式 SDC(Sno, Sname, Sdept, Sdean, Cno, Grade)^a

- Sno: 学生学号
- Sname: 学生姓名
- Sdept: 学生所在的系
- Sdean: 学生所在系的系主任
- Cno: 学生选修的课程的课号
- Grade: 学生选修这门课程的成绩

关系模式 SDC 的候选键是 (Sno, Cno)

^aSDC 代表 Student-Department-Course

问题 1 (数据插入异常)

该插入的数据无法插入

例 (数据插入异常)

- 操作：向 SDC 中插入一条学生元组，但该学生尚未选课
- 结果：由于该学生尚未选课，该元组的 Cno 属性值为空，违反实体完整性约束，因此不能插入该元组

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics	Einstein	1002	92
PH-001	Nick	Physics	Einstein	2003	85
PH-001	Nick	Physics	Einstein	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100
MA-002	Cindy	Math	Gauss		

问题 2 (数据删除异常)

不该删除的数据不得不删除

例 (数据删除异常)

- 操作: Ed 只选了一门课, 但他现在连这门课也不选了, 需要删除选课信息
- 结果: 由于 Cno 是 SDC 的主属性, 取值不能为空, 因此整个 Ed 元组都要被删除

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics	Einstein	1002	92
PH-001	Nick	Physics	Einstein	2003	85
PH-001	Nick	Physics	Einstein	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100

问题 3 (数据修改繁琐)

数据修改繁琐且易出错

例 (数据修改繁琐)

- 操作: Nick 转到 CS 系后, 需要修改他所在系和系主任的信息
- 结果: Nick 选了 3 门课, 因此必须修改这 3 条元组的 Sdept 和 Sdean 属性值, 否则会导致数据不一致

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics CS	Einstein Turing	1002	92
PH-001	Nick	Physics CS	Einstein Turing	2003	85
PH-001	Nick	Physics CS	Einstein Turing	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100

问题 4 (数据冗余)

存在大量冗余数据

例 (数据冗余)

- 如果一名学生选了 n 门课, 则他的 Sname、Sdept 和 Sdean 属性值都要重复存储 n 次

Sno	Sname	Sdept	Sdean	Cno	Grade
PH-001	Nick	Physics	Einstein	1002	92
PH-001	Nick	Physics	Einstein	2003	85
PH-001	Nick	Physics	Einstein	3006	88
CS-001	Elsa	CS	Turing	3006	90
CS-001	Elsa	CS	Turing	2003	95
CS-002	Ed	CS	Turing	2003	80
MA-001	Abby	Math	Gauss	1002	100

解决方法

关系模式分解

例 (关系模式分解)

把 SDC 分解为两个关系模式 SD(Sno, Sname, Sdept, Sdean) 和 SC(Sno, Cno, Grade)

$SD = \Pi_{Sno, Sname, Sdept, Sdean}(SDC)$

Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss

$SC = \Pi_{Sno, Cno, Grade}(SDC)$

Sno	Cno	Grade
PH-001	1002	92
PH-001	2003	85
PH-001	3006	88
CS-001	3006	90
CS-001	2003	95
CS-002	2003	80
MA-001	1002	100

关系模式分解

解决了数据插入异常问题

例 (解决数据插入异常问题)

- 操作: 向 SDC 中插入一条学生元组, 但该学生尚未选课
- 结果: 将该学生的信息插入到 SD 中, 不存在插入异常

SD				SC		
Sno	Sname	Sdept	Sdean	Sno	Cno	Grade
PH-001	Nick	Physics	Einstein	PH-001	1002	92
CS-001	Elsa	CS	Turing	PH-001	2003	85
CS-002	Ed	CS	Turing	PH-001	3006	88
MA-001	Abby	Math	Gauss	CS-001	3006	90
MA-002	Cindy	Math	Gauss	CS-001	2003	95
				CS-002	2003	80
				MA-001	1002	100

关系模式分解

解决了数据删除异常问题

例 (解决数据删除异常问题)

- 操作: Ed 只选了一门课, 但他现在连这门课也不选了, 需要删除选课信息
- 结果: 只需将 Ed 的选课信息从 SC 中删除, Ed 在 SD 中的元组不受影响, 因此不存在删除异常

SD				SC		
Sno	Sname	Sdept	Sdean	Sno	Cno	Grade
PH-001	Nick	Physics	Einstein	PH-001	1002	92
CS-001	Elsa	CS	Turing	PH-001	2003	85
CS-002	Ed	CS	Turing	PH-001	3006	88
MA-001	Abby	Math	Gauss	CS-001	3006	90
				CS-001	2003	95
				CS-002	2003	80
				MA-001	1002	100

关系模式分解

简化了数据修改

例 (简化数据修改)

- 操作: Nick 转到 CS 系后, 需要修改他所在的系和系主任的信息
- 结果: 只需在 SD 中将 Nick 所在的系和系主任的信息修改 1 次, 简化了修改操作

SD				SC		
Sno	Sname	Sdept	Sdean	Sno	Cno	Grade
PH-001	Nick	Physics	Einstein	PH-001	1002	92
		CS	Turing	PH-001	2003	85
CS-001	Elsa	CS	Turing	PH-001	3006	88
CS-002	Ed	CS	Turing	CS-001	3006	90
MA-001	Abby	Math	Gauss	CS-001	2003	95
				CS-002	2003	80
				MA-001	1002	100

关系模式分解

减少了数据冗余

例 (减少数据冗余)

- 无论一名学生选了几门课, 该学生的姓名、所在系和系主任的信息只需在 SD 中存储 1 次, 因此减少了数据冗余

SD				SC		
Sno	Sname	Sdept	Sdean	Sno	Cno	Grade
PH-001	Nick	Physics	Einstein	PH-001	1002	92
CS-001	Elsa	CS	Turing	PH-001	2003	85
CS-002	Ed	CS	Turing	PH-001	3006	88
MA-001	Abby	Math	Gauss	CS-001	3006	90
				CS-001	2003	95
				CS-002	2003	80
				MA-001	1002	100

问题 1: 数据插入异常

该插入的数据无法插入

例 (数据插入异常)

- **操作:** AI 系刚成立, Minsky 担任系主任, 尚无学生。向数据库中插入上述信息
- **结果:** 因为 SD 的主键是 (Sno), 而该元组的 Sno 属性值为空, 违反了实体完整性约束, 所以无法把该元组插入 SD

SD			
Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss
		AI	Minsky

问题 2: 数据删除异常

不该删除的数据不得不删除

例 (数据删除异常)

- **操作:** CS 系的学生全部毕业了, 删除 CS 系全体学生的元组
- **结果:** 在删除 CS 系全体学生元组的同时, 该系及系主任的信息也被删除了

SD			
Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss

问题 3: 数据修改繁琐

数据修改繁琐且易出错

例 (数据修改繁琐)

- **操作:** CS 系的系主任更换为 Shannon
- **结果:** 必须修改 CS 系全体学生元组的 Sdean 属性值, 否则会导致数据不一致

SD			
Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing Shannon
CS-002	Ed	CS	Turing Shannon
MA-001	Abby	Math	Gauss

问题 4: 数据冗余

存在大量冗余数据

例 (数据冗余)

一个系有多少名学生, 这个系的系主任的姓名就重复记录多少次

SD			
Sno	Sname	Sdept	Sdean
PH-001	Nick	Physics	Einstein
CS-001	Elsa	CS	Turing
CS-002	Ed	CS	Turing
MA-001	Abby	Math	Gauss

解决方法

关系模式分解

例 (关系模式分解)

把 SD 分解为两个关系模式 S(Sno, Sname, Sdept) 和 D(Sdept, Sdean)

$S = \Pi_{Sno, Sname, Sdept}(SD)$

Sno	Sname	Sdept
PH-001	Nick	Physics
CS-001	Elsa	CS
CS-002	Ed	CS
MA-001	Abby	Math

$D = \Pi_{Sdept, Sdean}(SD)$

Sdept	Sdean
Physics	Einstein
CS	Turing
Math	Gauss

关系模式分解

解决了数据插入异常问题

例 (解决数据插入异常问题)

- 操作: AI 系刚成立, Minsky 担任系主任, 尚无学生。向数据库中添加上述信息
- 结果: 向 D 中插入元组 (AI, Minsky), 没有插入异常问题

$S = \Pi_{Sno, Sname, Sdept}(SD)$

Sno	Sname	Sdept
PH-001	Nick	Physics
CS-001	Elsa	CS
CS-002	Ed	CS
MA-001	Abby	Math

$D = \Pi_{Sdept, Sdean}(SD)$

Sdept	Sdean
Physics	Einstein
CS	Turing
Math	Gauss
AI	Minsky

关系模式分解

解决了数据删除异常问题

例 (解决数据删除异常问题)

- 操作: CS 系的学生全部毕业了, 删除 CS 系全体学生的元组
- 结果: 从 S 中删除 CS 系全体学生元组, 该系及系主任的信息仍然保存在 D 中, 没有删除异常问题

$S = \Pi_{Sno, Sname, Sdept}(SD)$

Sno	Sname	Sdept
PH-001	Nick	Physics
CS-001	Elsa	CS
CS-002	Ed	CS
MA-001	Abby	Math

$D = \Pi_{Sdept, Sdean}(SD)$

Sdept	Sdean
Physics	Einstein
CS	Turing
Math	Gauss

关系模式分解

简化数据修改

例 (简化数据修改)

- 操作: CS 系的系主任更换为 Shannon
- 结果: 修改 D 中 CS 系元组的 Sdean 属性值, 仅修改 1 条元组, 简化了修改过程

$S = \Pi_{Sno, Sname, Sdept}(SD)$

Sno	Sname	Sdept
PH-001	Nick	Physics
CS-001	Elsa	CS
CS-002	Ed	CS
MA-001	Abby	Math

$D = \Pi_{Sdept, Sdean}(SD)$

Sdept	Sdean
Physics	Einstein
CS	Turing Shannon
Math	Gauss

关系模式分解

减少数据冗余

例 (减少数据冗余)

- 无论一个系有多少名学生, 该系的系主任信息只存储 1 次, 减少了数据冗余

$S = \Pi_{Sno, Sname, Sdept}(SD)$

Sno	Sname	Sdept
PH-001	Nick	Physics
CS-001	Elsa	CS
CS-002	Ed	CS
MA-001	Abby	Math

$D = \Pi_{Sdept, Sdean}(SD)$

Sdept	Sdean
Physics	Einstein
CS	Turing
Math	Gauss

关系模式规范化的优点和缺点

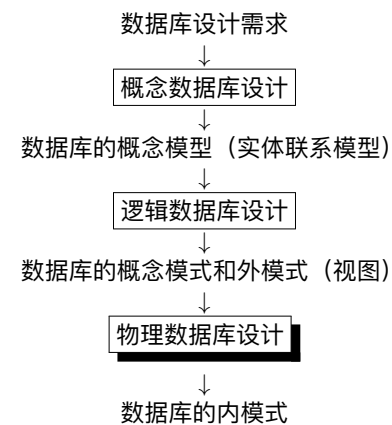
关系模式越规范, 异常越少:-)

↑
关系模式分解

↓
关系越多, 查询时需要做的连接操作越多:-)

11.4 物理数据库设计

数据库设计的过程



物理数据库设计 (Physical Database Design)

- 数据库内模式设计：在逻辑数据库设计的基础上，为数据库中的关系选择合适的存储结构和存取方法 (access method)
- 数据库调优 (database tuning)：使数据库上的事务能够高效执行，满足用户对数据库系统的性能需求

11.4.1 工作负载分析

物理数据库设计的任务

- 工作负载分析：分析工作负载中影响物理数据库设计的因素
- 索引设计：为关系选择合适的存取方法（主要是索引）
- 内模式设计：设计数据库的物理存储结构
- 查询改写：改写低效查询
- 概念模式调优：从提高数据库性能（而非规范性）的角度，调整数据库的概念模式

工作负载 (Workload)

定义 (工作负载)
一组混合在一起的查询和更新

查询列表

- 查询语句
- 查询出现的频率
- 用户对查询性能的要求

更新列表

- 更新语句
- 更新的出现频率
- 用户对更新性能的要求

查询语句分析

对于工作负载中每个查询语句，分析：

- 查询涉及的关系
- 投影属性（SELECT 子句涉及的属性）
- 选择条件和连接条件涉及的属性
- 选择条件和连接条件的选择度（Selectivity）

定义（选择度（Selectivity））

满足条件的结果元组在全部候选元组中所占的比例

更新语句分析

对于工作负载中每个更新语句，分析：

- 被更新的关系
- 更新的类型（插入、修改、删除）
- 被修改的关系上被修改的属性
- 更新条件涉及的关系
- 选择条件和连接条件涉及的属性
- 选择条件和连接条件的选择度

11.4.2 索引设计

索引设计的决策

- ① 在哪个关系上创建索引？
- ② 索引键包含什么属性？
- ③ 索引支持什么类型的查询（等值查询、区间查询）？
- ④ 索引结构是什么（B+ 树、哈希表）？
- ⑤ 索引是聚簇索引还是非聚簇索引？

索引设计的基本过程

依次检查工作负载中每个重要的查询 Q（包括更新操作的查询部分）

- 1 获取查询优化器为 Q 制定的查询执行计划
- 2 判断能否通过创建新的索引 I 而获得更高效的查询计划
- 3 如果可以，并且索引 I 的更新不会显著降低工作负载中更新操作的效率，则将索引 I 作为候选

索引设计的准则

索引设计准则 1（是否建索引）

- 只需在查询涉及的关系上创建索引
- 如果工作负载无法获益，则不创建索引
- 创建一个索引最好能加速多个查询

索引设计的准则

索引设计准则 2（索引键的确定）

- 索引键只需在连接条件和选择条件涉及的属性中选择
- 等值选择条件 $\Rightarrow$ 选择属性上的哈希表
- 区间选择条件 $\Rightarrow$ 选择属性上的 B+ 树

例（确定索引键）

- 1 `SELECT * FROM Student WHERE Sname = 'Elsa';`
`CREATE INDEX idx_sname ON Student (Sname) USING HASH;`
- 2 `SELECT * FROM Student WHERE Sage BETWEEN 18 AND 19;`
`CREATE INDEX idx_sage ON Student (Sage) USING BTREE;`

索引设计的准则

索引设计准则 3（复合索引的设计）

- WHERE 子句中包含同一关系中多个属性上的选择条件 $\Rightarrow$ 复合索引（Composite Index, 多个属性上的索引）
- 包含区间选择条件 $\Rightarrow$ 复合索引键中属性的顺序
- 考虑设计覆盖索引（Covering Index）

例（设计复合索引）

- 1 `SELECT * FROM Student`
`WHERE Ssex = 'M' AND Sdept = 'CS';`
`CREATE INDEX idx_ssex_sdept ON Student (Sdept, Ssex);`
- 2 `SELECT * FROM Student`
`WHERE Sdept = 'CS' AND Sage BETWEEN 18 AND 19;`
`CREATE INDEX idx_sdept_sage ON Student (Sdept, Sage);`

B+ 树支持的查询 I

例 (B+ 树支持的查询)

```
CREATE INDEX idx_sname_sage_ssex
ON Student (Sname, Sage, Ssex)
USING BTREE;
```

Sname	Sage	Ssex	RID
Cindy	19	F	rid1
Ed	18	M	rid2
Elsa	19	F	rid3
Elsa	19	M	rid4
Elsa	20	F	rid5
Fawn	18	F	rid6

❶ 查询涉及与所有索引键属性的等值比较

```
SELECT * FROM Student
WHERE Sname = 'Elsa' AND Sage = 19 AND Ssex = 'F';
```

❷ 查询涉及与索引键前缀属性的等值比较

```
SELECT * FROM Student WHERE Sname = 'Elsa' AND Sage = 19;
```

Navigation icons

B+ 树支持的查询 II

❸ 查询涉及与索引键最左属性的值的前缀的等值比较

```
SELECT * FROM Student WHERE Sname LIKE 'E%';
```

❹ 查询涉及对索引键最左属性的范围查找

```
SELECT * FROM Student WHERE Sname BETWEEN 'Ed' AND 'Emma';
```

❺ 查询涉及与索引键前 k 个属性的等值比较, 以及对第 $k+1$ 个属性的范围查找

```
SELECT * FROM Student
WHERE Sname = 'Elsa' AND Sage BETWEEN 18 AND 20;
```

❻ 查询结果排序属性列表是索引键的前缀, 且排序顺序相同

```
SELECT * FROM Student ORDER BY Sname, Sage;
```

Navigation icons

索引调优 (Index Tuning)

重建索引

- 索引可能因系统故障而损坏
- 索引在反复更新后可能出现空间碎片, 导致索引变得“臃肿”
- 修改了索引的参数 (如填充因子), 并希望这种修改完全生效
- 使用 **REINDEX** 命令重建索引 (PostgreSQL 和 openGauss)

定期更新数据库的统计信息

- 查询优化器根据代价估计模型估计的代价决定是否使用索引
- 过时的统计信息会降低代价估计模型的准确性
- 使用 **ANALYZE** 命令收集数据库的统计信息

Navigation icons

11.4.3 查询改写

Navigation icons

查询改写 (Query Rewrite)

如果通过改写查询就能获得好的查询计划，则没必要增加新的索引

- 查询优化器不能保证总是找到好的查询计划
- 用户基本上无法给 DBMS 指定查询计划
- 用户可以通过添加索引或改写查询来影响查询优化器的决策

查询改写的案例 1

例 (恒真/假的选择条件)

改写前:

- 1 `SELECT * FROM Student WHERE 1 = 1;`
- 2 `SELECT * FROM Student WHERE 1 = 0;`

改写原因:

- 选择条件恒为真/假
- 大量 CPU 资源浪费在过滤元组上

改写后:

- 1 `SELECT * FROM Student;`
- 2 无结果

查询改写的案例 2

例 (含表达式的选择条件)

改写前:

```
SELECT * FROM Student WHERE 2026 - Sage = 2007;
```

改写原因:

- 选择条件含有表达式
- 查询优化器无法确定使用哪个索引

改写后:

```
SELECT * FROM Student WHERE Sage = 19;
```

查询改写的案例 3

例 (含 OR 的选择条件)

改写前:

```
SELECT Sno, Sname FROM Student WHERE Ssex = 'F' OR Sage = 19;
```

改写原因:

- 没有索引可以支持该选择条件
- 如果在 Ssex 和 Sage 上分别建有索引，则改写查询可能使 DBMS 使用索引合并 (Index Merge)

改写后:

```
(SELECT Sno, Sname FROM Student WHERE Ssex = 'F') UNION  
(SELECT Sno, Sname FROM Student WHERE Sage = 19);
```

查询改写的案例 4

例 (无用的去重操作)

改写前:

```
SELECT DISTINCT Sno, Sname FROM Student WHERE Ssex = 'F';
```

改写原因:

- 元组去重操作很昂贵
- 因为 SELECT 子句中包含候选键 Sno, 所以 DISTINCT 画蛇添足

改写后:

```
SELECT Sno, Sname FROM Student WHERE Ssex = 'F';
```

查询改写的案例 5

例 (无用的分组操作)

改写前:

```
SELECT MIN(Sage) FROM Student GROUP BY Sdept HAVING Sdept = 'CS';
```

改写原因:

- 分组操作很昂贵
- 查询中的分组操作毫无必要

改写后:

```
SELECT MIN(Sage) FROM Student WHERE Sdept = 'CS';
```

查询改写的案例 6

例 (无用的投影操作)

改写前:

```
SELECT * FROM SC WHERE EXISTS (  
    SELECT Sno FROM Student  
    WHERE Sname = 'Elsa' AND Student.Sno = SC.Sno);
```

改写原因:

- EXISTS 只检查子查询结果是否为空
- 子查询的投影操作没必要

改写后:

```
SELECT * FROM SC WHERE EXISTS (  
    SELECT * FROM Student  
    WHERE Sname = 'Elsa' AND Student.Sno = SC.Sno);
```

查询改写的案例 7

例 (无用的连接操作)

改写前:

```
SELECT Sno, Cno, Grade FROM Student NATURAL JOIN SC;
```

改写原因:

- SC 中每条元组一定能够与 Student 中的元组连接
- SELECT 子句中的属性全部来自 SC, 故连接操作没有必要

改写后:

```
SELECT * FROM SC;
```

查询改写的案例 8

例 (临时关系)

改写前:

```
SELECT * INTO Temp FROM Student NATURAL JOIN SC WHERE Sdept = 'CS';
```

```
SELECT Cno, MAX(Grade) FROM Temp GROUP BY Cno;
```

改写原因:

- 物化临时关系 Temp 占用额外的存储空间
- 临时关系 Temp 上没有索引, 可能会浪费掉潜在的查询优化机会 (假设已有 SC(Cno, Grade) 上的复合索引)

改写后:

```
SELECT Cno, MAX(Grade) FROM Student NATURAL JOIN SC  
WHERE Sdept = 'CS' GROUP BY Cno;
```

查询改写的案例 9

例 (嵌套查询)

改写前:

```
SELECT Cno FROM SC  
WHERE Sno IN (SELECT Sno FROM Student WHERE Sdept = 'CS');
```

改写原因:

- 查询优化器优化嵌套查询的能力通常较差
- 将嵌套查询改写为非嵌套查询通常会带来更优的查询执行计划

改写后:

```
SELECT Cno FROM Student NATURAL JOIN SC WHERE Sdept = 'CS';
```

查询改写的案例 10

例 (类型转换)

改写前:

```
SELECT * FROM Student WHERE Sno = 123456;
```

改写原因:

- Student 关系的 Sno 属性是字符串型
- DBMS 不会使用 Sno 上的索引, 而是使用顺序扫描
- 因为 Sno 属性值为字符串型, 需要转换为整型

改写后:

```
SELECT * FROM Student WHERE Sno = '123456';
```

总结

- 1 11.1 数据库设计的过程
- 2 11.2 概念数据库设计
 - 11.2.1 与实体相关的概念
 - 11.2.2 与联系相关的概念
- 3 11.3 逻辑数据库设计
 - 11.3.1 ER 模型转换为关系数据库模式
 - 11.3.2 关系模式规范化
- 4 11.4 物理数据库设计
 - 11.4.1 工作负载分析
 - 11.4.2 索引设计
 - 11.4.3 查询改写

致谢

2026 版以前

- 课件中的 ER 图使用 Tikz 绘制²
- 感谢李一鸣同学指出了课件中的错误

²<http://www.texample.net/tikz/examples/er-diagram/>